Semantic Parsing

Understanding Language Through Meaning Representation

Introduction

- Semantic Parsing deals with understanding natural language by converting text into a meaningful representation.
- Two main approaches have evolved in Natural Language Processing (NLP) for language understanding.

Approach 1 — Domain-Specific Representation

- Creates a rich meaning representation for a limited domain.
- Used in applications like travel reservations, football simulations, and geographic databases.
- Known as Deep Semantic Parsing.

Advantages & Limitations of Approach 1

- Advantages: Produces deep understanding for specific applications, high accuracy within its domain.
- Limitations: Poor reusability across domains, adapting to new domains requires major rework.

Approach 2 — Intermediate Representations

- Focuses on creating a series of intermediate meaning layers.
- Breaks complex understanding into smaller, manageable tasks: word sense disambiguation, predicate–argument structure recognition.
- Known as Shallow Semantic Parsing.

Characteristics of Shallow Semantic Parsing

- Produces general-purpose, low-level representations.
- Easier to train and apply across multiple tasks.
- However, building a general-purpose ontology that works everywhere is challenging.

Ontology — The Core Challenge

Ontology means:

1. A branch of metaphysics dealing with the nature of being.

2. In NLP, it refers to a set of concepts and relationships within a domain.

"What’s new about our ontology is that it is created automatically from large datasets."

Need for a Translation Layer

- A translation layer is needed to connect the general (shallow) and specific (deep) representations.
- Ensures that general models can support domain-specific applications.

Summary

Two key types of semantic parsing:

- Deep Semantic Parsing — Detailed, domain-specific.
- Shallow Semantic Parsing — General-purpose, intermediate.

Balancing depth, reusability, and generality remains a core challenge in NLP.

Semantic Interpretation

Understanding the Structure and Components of Semantic Parsing

1. Introduction to Semantic Interpretation

- Semantic parsing is part of Semantic Interpretation.
- It defines a representation of text that can be processed by computers.
- It enables further computation, manipulation, and search for understanding natural language.

2. Requirements of a Semantic Theory

A Semantic theory should be able to:

➢ Explain ambiguous sentences (e.g., 'The bill is large').

➢ Resolve word ambiguities in context.

➢ Identify meaningless but syntactically correct sentences.

➢ Identify paraphrases with the same meaning.

2.1. Structural Ambiguity

➢ Refers to ambiguity arising from the syntactic structure of a sentence.

➢ Semantic interpretation relies on understanding underlying syntactic structures.

2.2. Word Sense

➢ The same word can have multiple meanings depending on context.

➢ Example: 'nail' can mean a body part or a metallic fastener.

➢ Word sense disambiguation helps identify the correct meaning.

2.3. Entity and Event Resolution

➢ Every discourse involves entities and events.

➢ The task is to identify entities across text using similar or different phrases.

➢ Includes Named Entity Recognition and Coreference Resolution.

➢ Coreference Resolution: finding all expressions that refer to the same entity.

2.4. Predicate-Argument Structure

➢ Once word senses, entities, and events are known, we identify their relationships.

➢ A predicate is typically the verb; arguments are the nouns.

➢ Example: identifying which entity performs which action in an event.

2.5. Meaning Representation

➢ The final step of semantic interpretation.

➢ Builds a meaning representation that can be used by applications.

➢ Also known as deep semantic representation.

➢ Enables reasoning, inference, and natural language understanding.

3. System Paradigms

- It is important to get a perspective on the various primary dimensions on which the problem of semantic interpretation has been tackled.
- The approaches generally fall into the following three categories:
  - 1.System architecture
  - 2.Scope
  - 3. Coverage.
  - 4.Word sense

1. System Architectures

a. Knowledge based: These systems use a predefined set of rules or a knowledge base to obtain a solution to a new problem.

b. Unsupervised: These systems tend to require minimal human intervention to be functional by using existing resources that can be bootstrapped for a particular application or problem domain.

c. Supervised: these systems involve the manual annotation of some phenomena that appear in a sufficient quantity of data so that machine learning algorithms can be applied.

d. Semi-Supervised: manual annotation is usually very expensive and does not yield enough data to completely capture a phenomenon. In such instances, researches can automatically expand the data set on which their models are trained either by employing machine-generated output directly or by bootstrapping off an existing model by having humans correct its output.

2. Scope

- **Domain Dependent**: These systems are specific to certain domains, such as air travel reservations or simulated football coaching.
- **Domain Independent**: These systems are general enough that the techniques can be applicable to multiple domains without little or no change.

3. Coverage

a. **Shallow**: These systems tend to produce an intermediate representation that can then be converted to one that a machine can base its action on.
b. **Deep:** These systems usually create a terminal representation that is directly consumed by a machine or application.

Word Sense

- **Word sense** means the *specific meaning* a word takes in a particular **context**.
- Many words are **polysemous** , they have **multiple related meanings** or
- **homonymous**, having *completely different meanings* but spelled the same.
- Example:
  - “He went to the bank to deposit money.”- A financial institution
  - “They had a picnic by the bank of the river.” -Edge or slope beside a river
  - “The pilot banked the plane to the left.” - Tilt or turn of a plane

- Correctly understanding the sense of words helps many NLP tasks:
  - Translation -To translate words correctly (e.g., “bank”)
  - Question Answering -To find the right answer matching the right meaning
  - Information Retrieval - To fetch documents with relevant meaning, not just keywords
  - Text Summarization - To interpret words correctly in summary
  - Sentiment Analysis Wrong sense may flip sentiment meaning (“cool app” vs. “cool weather”)

Word Sense Disambiguation (WSD)

- is the process of identifying which sense of a word is used in a given context.
- Formally:

  WSD = f(word, context) → correct_sense(word)
- For example:

  WSD("bank", "He went to the bank to deposit money.") → financial institution

Approaches to WSD

A. Knowledge-Based Approaches:

These rely on predefined lexical databases like WordNet, which contains:

- Word senses (called synsets)
- Definitions (called glosses)
- Semantic relationships (synonymy, hypernymy, etc.)
- Algorithms:
  - Lesk Algorithm (1986)
  - Extended Lesk Algorithm
  - Semantic Similarity-Based

B. Supervised Learning Approaches:

- WSD is treated as a classification problem
- Each occurrence of an ambiguous word = one instance
- Label = correct WordNet sense
- Features used:
  - Surrounding words (context window)
  - Part-of-speech tags
  - Collocations
  - Morphological patterns
- Algorithms:
  - Naïve Bayes, SVM, Decision Trees, Neural Networks, etc.

- **C. Unsupervised / Semi-Supervised Approaches**
- No labeled data needed.
- They **cluster** word occurrences by context similarity:
- Each cluster → one sense
- Each instance → vector representation (TF-IDF or word embedding)
- **Example:**
  - Use Word2Vec embeddings for all contexts of “bank”
  - Apply K-Means clustering
  - Label clusters manually or automatically

- **D. Deep Learning and Contextual Embeddings**
  - Modern NLP models (like **BERT**, **RoBERTa**, **GPT**) *implicitly learn* word senses.
  - Each word embedding depends on its context → different sense = different vector.
- **E. Knowledge-Enhanced LLMs (2023–2025 trend)**
  - Recent works enhance WSD using **Large Language Models (LLMs)** with:
  - Chain-of-thought reasoning (e.g., “What is the meaning of ‘bank’ in this context?”)
  - Retrieval-Augmented Generation (RAG)
  - Integration with external knowledge bases (WordNet, ConceptNet)

What is Lesk Algorithm?

- Proposed by Michael Lesk (1986)
- A dictionary-based approach to WSD
- Uses overlap between dictionary definitions (glosses)
- The sense with the highest overlap with the context is chosen

Lesk Algorithm Steps

1. Input: Target word and context
2. Retrieve all possible dictionary definitions (glosses)
3. Retrieve glosses for surrounding context words
4. Compute overlap between glosses
5. Choose the sense with the maximum overlap

Example: Disambiguating 'bank'

- Sentence: “He deposited money in the bank.”
- Possible Senses:
  - 1. Financial institution that accepts deposits
  - 2. Land beside a river

- Overlap with context words (deposit, money):
  - Sense 1: Overlap = 2
  - Sense 2: Overlap = 0

  Chosen Sense: Financial Institution

Variants of Lesk Algorithm

- Original Lesk (1986): Uses dictionary glosses
- Simplified Lesk: Compares gloss with actual sentence context
- Extended Lesk: Expands glosses using WordNet relations (synonyms, hypernyms, etc.)

Advantages of Lesk Algorithm

- Simple and easy to implement
- Does not require training data
- Explainable and interpretable results

Drawbacks of Lesk Algorithm

- Relies heavily on dictionary quality
- Sensitive to short or incomplete glosses
- Does not consider syntax or deep semantics
- Struggles with ambiguous context words

Extended Lesk Algorithm

- The original Lesk algorithm (1986) is simple but limited.
- It only compares:
- the gloss (definition) of the target word's sense with the glosses of the immediate context words.
- However:
  - Dictionary glosses are often short → few overlaps.
  - Many semantically related words (e.g., finance, money, account) may not have exact lexical matches.
  - The algorithm does not use semantic relations (like hypernyms, hyponyms, meronyms).
- To overcome these, the Extended Lesk algorithm was proposed by Banerjee & Pedersen (2002) using WordNet.

- The **Extended Lesk Algorithm** expands the comparison to include **glosses of related senses** — not just the main gloss of the word.
- It compares the glosses of:
  - The **target word's sense**
  - Its **related senses** (like hypernyms, hyponyms, meronyms, etc.)
  - The **context word's senses** and their related senses.

- Hypernym:
  - Superclass (is-a relation) Dog → Animal
- Hyponym
  - Subclass (kind-of relation) Animal → Dog
- Meronym
  - Part of Wheel → Car
- Holonym
  - Whole of Car → Wheel

Including these relations gives a **richer semantic context** for overlap comparison.

- Steps:
- Step 1 — Choose the target wordExample sentence: “He went to the bank to deposit money.”
- Target word: bank
- Step 2 — Get possible senses (WordNet synsets)
- Sense - bank
- $Gloss_1$ -“A financial institution that accepts deposits and lends money.”
- Gloss2- “The land alongside or sloping down to a river or lake.”

- **Step 3 — Collect glosses for each related sense**
  - For each sense (e.g., *bank*$_1$), include glosses of its related words:
  - **Hypernyms** (e.g., “financial institution”, “organization”)
  - **Hyponyms** (e.g., “commercial bank”, “savings bank”)
  - **Meronyms** (e.g., “bank building”)
  - Do the same for context words like *deposit*, *money*.

- Step 4 — Compute overlaps between all gloss combinations
  - For each sense of bank, compute overlaps with:
    - Glosses of each sense of each context word,
    - and their related glosses (hypernyms, hyponyms, etc.).

Target	Context	Gloss Overlap Words
$bank_1$ (financial institution)	deposit	deposit, money
$bank_1$ (financial institution)	money	money, finance
$bank_2$ (river side)	deposit	none
$bank_2$ (river side)	money	none

- **Step 5 — Choose the sense with the highest overlap score**
  - $bank_1$ (financial sense): overlap = 4
  - $bank_2$ (river sense): overlap = 0
- Predicted sense: **$bank_1$ (financial institution)**

Walker’s Algorithm

A Semantic Similarity-Based Approach for Word Sense Disambiguation

Overview

- Walker’s Algorithm is a semantic similarity-based method for Word Sense Disambiguation (WSD).
- It uses lexical databases like WordNet to measure semantic closeness between word senses.
- The goal is to select the combination of senses with the highest overall similarity.

Main Idea

- Words can have multiple meanings (senses) organized in a hierarchy in WordNet.
- The algorithm compares all possible sense combinations between words.
- It chooses the combination with the smallest semantic distance (or highest similarity).

Steps of Walker's Algorithm

1. Input a sentence containing ambiguous words.
2. Retrieve all possible senses of each word from WordNet.
3. Compute pairwise semantic distances between all sense combinations.
4. Aggregate the distances.
5. Select the sense configuration that minimizes total distance.

Example

Sentence: 'The plant near the bank.'

Possible Senses:

- plant → {factory, living organism}
- bank → {financial institution, river side}

Distance Calculation:

- living organism – river side → low distance (natural context)
- factory – financial institution → high distance (different domains)

Correct senses: plant = living organism, bank = river side

Predicate–Argument Structure in Natural Language Processing

Introduction to Predicate–Argument Structure (PAS)

Predicate–Argument Structure (PAS) represents the underlying meaning of a sentence by identifying:

- Predicate – the main action (usually a verb)
- Arguments – entities involved in the action (subject, object, etc.)

Goal: Capture semantic relationships such as who did what to whom, when, and where.

Example: Understanding PAS

- Sentence: 'Alice gave Bob a book.'

- Predicate: gave
- Arguments:
  - Arg0 (Agent): Alice
  - Arg1 (Recipient): Bob
  - Arg2 (Theme): a book

- Representation: give(Alice, Bob, book)

Why Predicate–Argument Structure Matters in NLP

- Helps machines go beyond syntax to capture meaning.
- Enables understanding of semantic roles in sentences.
- Supports NLP tasks like Information Extraction, Question Answering, and Machine Translation.

Example: In QA, identifying the predicate helps link questions to answers.

Formal Representation of PAS

- PAS can be represented as predicate(arg1, arg2, ...)
- Example: hire(company, engineer, last_week)

- This structure is language-independent and abstracts away grammatical variations.

Applications of PAS in NLP

- Information Extraction – identify relations between entities.
- Machine Translation – maintain semantic equivalence across languages.
- Question Answering – extract relevant predicate-argument pairs.
- Summarization – capture key meaning units from text.

Challenges in Predicate–Argument Analysis

- Ambiguity – multiple meanings for the same predicate.
- Missing Arguments – implicit or context-based roles.
- Domain Variation – verbs and arguments differ across domains.
- Cross-linguistic Differences – structures vary in non-English languages.

- There are two resources
  - FrameNet
  - PropBank

FrameNet

- FrameNet is a lexical database based on Frame Semantics.
- Developed at the International Computer Science Institute (ICSI), Berkeley.
- Words evoke 'frames' — conceptual structures representing events or situations.
- Each frame includes participants (Frame Elements) and their roles.

Understanding Frame Semantics

- A 'frame' is a conceptual structure describing a type of event, relation, or object.

Example: The 'Buying' frame includes Buyer, Seller, Goods, and Money.

- Words like 'buy', 'purchase', 'sell', 'pay' all evoke the same frame.
- Connects word meaning with world knowledge context.

Frame Elements (FEs)

- Frame Elements (FEs) represent semantic roles in a frame.
- Core FEs are essential participants (Buyer, Goods).
- Non-core FEs give additional context (Time, Location).
- Example:

  Buyer: John | Goods: a car | Seller: dealer | Money: $10,000.

FrameNet Annotation Structure

- Each annotated sentence links lexical units to frames.

• Example:

Sentence: 'John bought a car from the dealer for $10,000.'

Buy(john,dealer,car,10000)

Frame: Buying | Buyer=John | Goods=a car | Seller=dealer | Money=$10,000.

• Used for training semantic role labeling (SRL) models.

Teach(teja,B.tech students, NLP, this night)

FrameNet Example Diagram

Sentence: 'Mary paid the shopkeeper for the apples.'

Evoked Frame: COMMERCIAL_TRANSACTION

Frame Elements:
- Buyer: Mary
- Seller: shopkeeper
- Goods: apples
- Money: (implicit)

Paid(mary, shopkeeper, apples)

Visual: Arrows linking each word to its Frame Element.

FrameNet Resources and Tools

- Berkeley FrameNet: https://framenet.icsi.berkeley.edu/
- OpenFrameNet API for Python.
- Used in SRL, QA, and multilingual frame semantics.
- Examples: FrameNet Brasil, Spanish FrameNet.

2. PropBank

- PropBank (Proposition Bank) is a corpus that adds a layer of semantic annotation over the Penn Treebank.
- It defines argument structures (predicate-argument relationships) for verbs.
- Developed at the University of Pennsylvania.
- Each verb in the corpus is associated with a set of roles, labeled as ARG0, ARG1, etc.

PropBank

- Penn Treebank captures syntax, but not semantic roles.
- PropBank bridges the gap between syntax and semantics.
- Enables Semantic Role Labeling (SRL) by specifying 'who did what to whom'.
- Facilitates NLP tasks like question answering, summarization, and information extraction.

Structure of PropBank Annotation

- Each verb is annotated with a frame file describing its argument structure.
- Example Frame File:

  Verb: give

  Roleset: give.01 – transfer of possession

  ARG0: giver

  ARG1: thing given

  ARG2: recipient
- Example sentence: 'John gave Mary a book.' → ARG0=John, ARG1=book, ARG2=Mary

Role Labels in PropBank

- ARG0: typically the agent or doer.
- ARG1: the patient, theme, or entity affected.
- ARG2, ARG3, ARG4: secondary roles like instrument, benefactive, or source.
- ARGM: modifiers – e.g., ARGM-TMP (time), ARGM-LOC (location), ARGM-MNR (manner).

Example Annotation in PropBank

- Sentence: 'The chef cooked dinner for the guests yesterday.'

- Verb: cook.01 – prepare food
- ARG0: The chef (agent)
- ARG1: dinner (thing cooked)
- ARG2: the guests (beneficiary)
- ARGM-TMP: yesterday (time)

- → Captures complete predicate-argument structure.

PropBank Tools and Resources

- Original corpus: http://propbank.github.io/
- Integrated in CoNLL-2005/2009 SRL shared tasks.
- Commonly used with NLTK, AllenNLP, SpaCy.
- Multilingual PropBanks: Chinese, Arabic, Hindi, etc.

Example Dataset Entry (Simplified)

- Sentence: 'The teacher explained the lesson to students clearly.'

- Verb: explain.01
- ARG0: The teacher
- ARG1: the lesson
- ARG2: students
- ARGM-MNR: clearly

- Provides labeled relationships for SRL training.

1. Semantic Roles and Argument Labels

- Core Arguments (ARG0 – ARG5)
- Each verb sense defines which arguments are valid and what they represent.

Label	Typical Role	Example
ARG0	Agent / Doer	“Mary” in “Mary sold a car.”
ARG1	Theme / Thing affected	“a car”
ARG2	Recipient / Beneficiary / Attribute	“to John”
ARG3	Starting point / Instrument / Secondary role	“for cash”
ARG4	Ending point / Result	“into pieces”
ARG5	Directional or other rare argument	Used for special cases

2. Adjunct Arguments (ARGM-xxx)

- Adjuncts (ARGM) represent modifiers — information not essential to the core meaning but giving additional context.

Label	Meaning	Example
ARGM-TMP	Temporal (time)	“yesterday”
ARGM-LOC	Location	“in Delhi”
ARGM-MNR	Manner	“quickly”
ARGM-CAU	Cause	“because of rain”
ARGM-DIR	Direction	“toward the gate”
ARGM-EXT	Extent	“by 20%”
ARGM-ADV	Adverbial	“frankly”
ARGM-NEG	Negation	“not”
ARGM-DIS	Discourse	“however,” “therefore”

FrameNet vs. PropBank

- PropBank: verb-specific argument structures.
- FrameNet: conceptual frames covering broader semantics.
- PropBank uses ARG labels; FrameNet uses semantic roles.
- Example:

  PropBank: ARG0=buyer, ARG1=thing bought.

  FrameNet: Buyer, Goods, Seller, Money.

- PropBank focuses on predicate-specific roles (syntactic and semantic).
- FrameNet focuses on conceptual frames and world knowledge.
- PropBank is easier to annotate automatically.
- FrameNet provides richer semantic generalization.
- Often both are used together for comprehensive SRL models.

1. Conceptual Basis

- PropBank: Predicate–argument structures centered around verbs.
- FrameNet: Based on Frame Semantics, where each word evokes a conceptual frame.
- PropBank = practical SRL corpus; FrameNet = conceptual knowledge model.

2. Representation of Meaning

- PropBank: Focuses on individual verbs and their arguments.
- FrameNet: Groups related words (buy, sell, pay) into frames describing events.

Example:

PropBank → give.01: ARG0=giver, ARG1=thing, ARG2=recipient

FrameNet → Commerce_buy frame: Buyer, Seller, Goods, Money.

3. Role Labels

- PropBank uses generic ARG roles (ARG0–ARG5).
- FrameNet uses named Frame Elements (Buyer, Seller, Goods, etc.).
- ARG roles are verb-specific; FEs are frame-consistent across verbs.

4. Data and Annotation

- PropBank built on Penn Treebank; annotations for each verb.
- FrameNet is a lexical database with annotated example sentences.
- PropBank easier to annotate; FrameNet requires linguistic expertise.

5. Example Comparison

- Sentence: 'Mary sold John a book.'

- PropBank:
  - Predicate: sell.01
  - ARG0: Mary (seller)
  - ARG1: book (goods)
  - ARG2: John (buyer)

- FrameNet:
  - Frame: Commerce_sell
  - Seller: Mary
  - Goods: book
  - Buyer: John

6. Coverage and Granularity

- PropBank: Broad coverage, shallow semantic detail.
- FrameNet: Narrow coverage, deep conceptual detail.
- FrameNet suitable for semantic reasoning, PropBank for SRL models.

7. Application in NLP

- Semantic Role Labeling (SRL): PropBank core dataset (CoNLL tasks).
- FrameNet supports conceptual SRL for deeper understanding.
- Used in Information Extraction, QA, and Machine Translation.

8. Relationship Between FrameNet and PropBank

- Complementary resources.
- PropBank: syntactic realization; FrameNet: semantic generalization.
- Combined use enhances NLP tasks like SRL and semantic parsing.

10. In Simple Words

- PropBank = Syntax-based labeling of ‘who did what to whom.’
- FrameNet = Knowledge-based representation of events and participants.
- Together, they power modern semantic understanding in NLP.

Meaning Representation Systems

- In NLP, meaning representation refers to the process of converting natural language sentences into a structured form that computers can interpret and reason about.
- The goal is to represent semantic meaning—not just syntax—so that machines can understand, infer, and generate human language.
- A meaning representation system is therefore a formal structure or language that expresses the meaning of natural language sentences.
- Example:

Sentence: John gave Mary a book.

Meaning representation (in logic): GIVE(John, Mary, Book)

Types of Meaning Representation Systems

- First-Order Predicate Logic (FOPL)
- Semantic Networks
- Conceptual Dependency (CD) Theory
- Frame-Based Representation
- Semantic Role Labeling (SRL)
- Abstract Meaning Representation (AMR)

(A) First-Order Predicate Logic (FOPL)

- The most widely used symbolic meaning representation.
- Represents facts as **predicates** with **arguments**.
- Supports logical inference using rules.

Example:

> Sentence: *All humans are mortal. Socrates is a human.*
> Representation:
> $\forall x$ [Human(x) $\rightarrow$ Mortal(x)]
> Human(Socrates)

Advantages:

- Clear semantics and strong reasoning ability.

Disadvantages:

- Hard to handle ambiguity and vagueness of natural language.

(B) Semantic Networks

- Graph-based representation of knowledge.
- Nodes represent **concepts or entities**; edges represent **relations**.

Example:

For the sentence *A cat is an animal*,

```
css

[Cat] → is_a → [Animal]
```

Advantages:

- Easy visualization; intuitive for humans.

Disadvantages:

- Difficult to perform deep reasoning; limited formal semantics.

(C) Conceptual Dependency (CD) Theory

- Developed by Roger Schank (1972).
- Represents meaning using **primitive conceptual acts** (like PTRANS = physical transfer).
- Focuses on actions and relationships rather than syntax.

Example:

> Sentence: *John gave a book to Mary*
> Representation: `(PTRANS (ACTOR John) (OBJECT Book) (TO Mary))`

Advantages:

- Language-independent representation of meaning.

Disadvantages:

- Complex to design and scale for all types of sentences.

(C) Conceptual Dependency (CD) Theory

- Developed by Roger Schank (1972).
- Represents meaning using **primitive conceptual acts** (like PTRANS = physical transfer).
- Focuses on actions and relationships rather than syntax.

Example:

> Sentence: *John gave a book to Mary*
> Representation: `(PTRANS (ACTOR John) (OBJECT Book) (TO Mary))`

Advantages:

- Language-independent representation of meaning.

Disadvantages:

- Complex to design and scale for all types of sentences.

(D) Frame-Based Representation

- Based on **Frame Semantics** (Fillmore, 1976).
- A **frame** represents a stereotypical situation (like buying, traveling).
- Each frame includes **frame elements** (roles) relevant to the event.

Example:

> Sentence: *Mary bought a laptop from John.*
> Frame: *Commerce_buy*
> Roles: Buyer = Mary, Seller = John, Goods = Laptop

Resources:

- **FrameNet** — annotated corpus based on frames.

Advantages:

- Captures rich semantic information.

(E) Semantic Role Labeling (SRL)

- Identifies **who did what to whom**, when, and where.
- Uses resources like **PropBank** and **FrameNet**.

Example:

> *John gave Mary a book.*
> Predicate: *gave*
> ARG0 (Giver) = John
> ARG1 (Thing given) = Book
> ARG2 (Recipient) = Mary

Advantages:

- Structured event representation; supports downstream NLP tasks.

Disadvantages:

- Relies on predefined verb frames.